

PRNN Programming Assignment 3

Arjun Mallick
Sr. 25989

1.1 Gini Impurity Reduction for a Continuous Feature

Since the dataset does not contain the `Temperature` feature, the `co` column is used.

Feature Statistics (`co`)

mean	2992.605783
min	270.370000
50%	1869.200000
max	21148.680000

Result

- Feature: `co`
- Optimal Threshold: 2790.45
- Gini Impurity Reduction: 0.282794

Conclusion

The feature `co` provides a significant impurity reduction at the root node, indicating strong discriminative power for separating high and low pollution levels. The optimal threshold effectively partitions the data into two groups with improved class purity.

1.2 Random Forest

A Random Forest classifier was implemented using multiple depth-limited decision trees with feature bagging.

Dataset Shapes

- Training data: (13143, 14)
- Test data: (2817, 14)

Result

- Test Accuracy: 97.27%

Conclusion

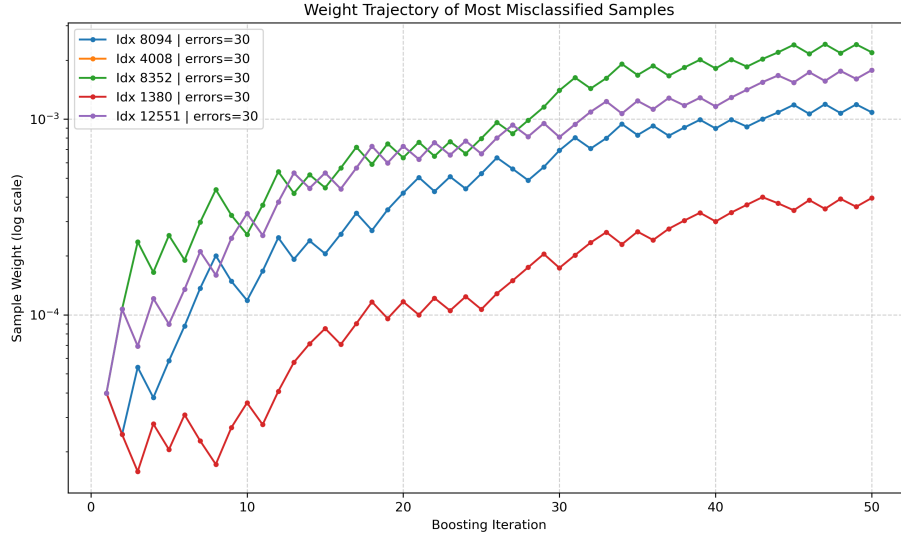
The model achieves high accuracy on the test set, indicating effective learning of patterns in the data. The ensemble approach improves robustness and generalization compared to a single tree.

1.3 AdaBoost Sample Reweighting

AdaBoost (AdaBoost.M1) was implemented using shallow decision stumps. The sample weights of the 5 most consistently misclassified training samples were tracked over 50 boosting iterations.

Results and Plot

The top 5 misclassified samples were identified based on cumulative misclassification counts. Their weights increase steadily across iterations.



Mathematical Explanation

At each boosting iteration, the sample weights are updated as:

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(x_i))$$

For misclassified samples:

$$\begin{aligned} y_i \neq h_t(x_i) &\Rightarrow y_i h_t(x_i) = -1 \\ \Rightarrow w_i^{(t+1)} &= w_i^{(t)} \cdot \exp(\alpha_t) \end{aligned}$$

Thus, each misclassification increases the weight multiplicatively by a factor of $\exp(\alpha_t)$. Over multiple iterations:

$$w_i \propto \exp\left(\sum_t \alpha_t\right)$$

This leads to exponential growth in weights for consistently misclassified samples, which appears approximately linear on a logarithmic scale, as observed in the plot.

1.4 Gradient Boosting Residuals (Regression)

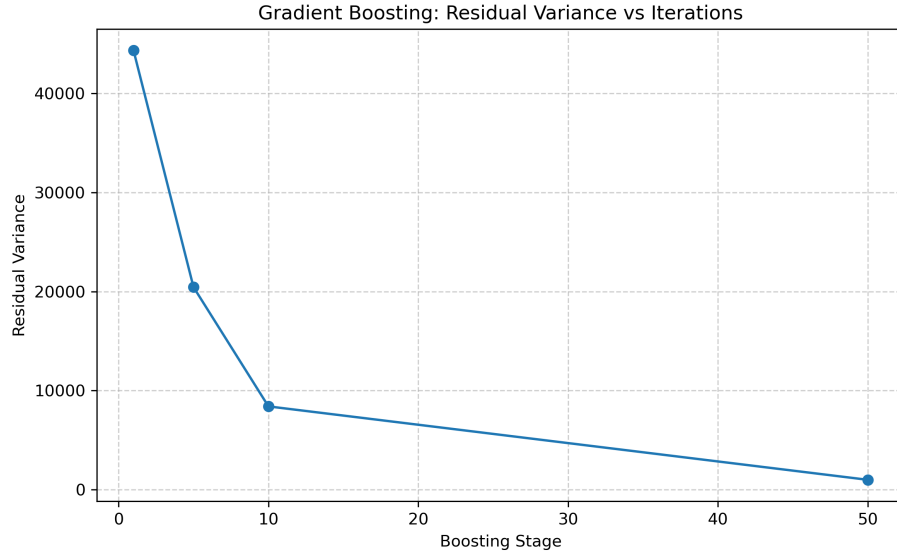
A Gradient Boosted Regressor was implemented using shallow trees trained on pseudo-residuals of the continuous pm2.5 target.

Result

Residual variance at selected boosting stages:

- Stage 1: 44324.06
- Stage 5: 20443.30
- Stage 10: 8402.96
- Stage 50: 991.44

Plot



Conclusion

The residual variance decreases significantly across boosting stages, showing that the model progressively reduces prediction errors. The rapid initial drop followed by slower improvement indicates diminishing returns with more iterations.

2.5 PCA vs. Linear Autoencoder

Method:

The 72-hour air quality sequences were flattened to 576-dimensional vectors. PCA was implemented using SVD, and the number of components required to retain 95% variance was computed. A purely linear autoencoder was trained with bottleneck dimension equal to this component count.

Result

- Number of components (95% variance): $k = 73$
- PCA Reconstruction MSE: 0.04988
- Autoencoder Reconstruction MSE: 0.05014

Conclusion

The autoencoder achieves reconstruction error comparable to PCA, indicating that both methods learn the same low-dimensional subspace. This empirically verifies that a purely linear autoencoder spans the same subspace as PCA.

2.6 GMM Latent Clustering

Method:

Feature embeddings were extracted from the validation set using the trained CNN by removing the classification head. A Gaussian Mixture Model (GMM) was fitted on these embeddings, and cluster assignments were obtained. The clustering performance was evaluated using the Adjusted Rand Index (ARI) with respect to the true labels.

Results:

- Adjusted Rand Index (ARI): 0.4659

Conclusion:

The obtained ARI indicates moderate alignment between the unsupervised clusters and the true class labels, suggesting that the learned CNN embeddings capture meaningful class-specific structure.

3.7 ELBO Implementation

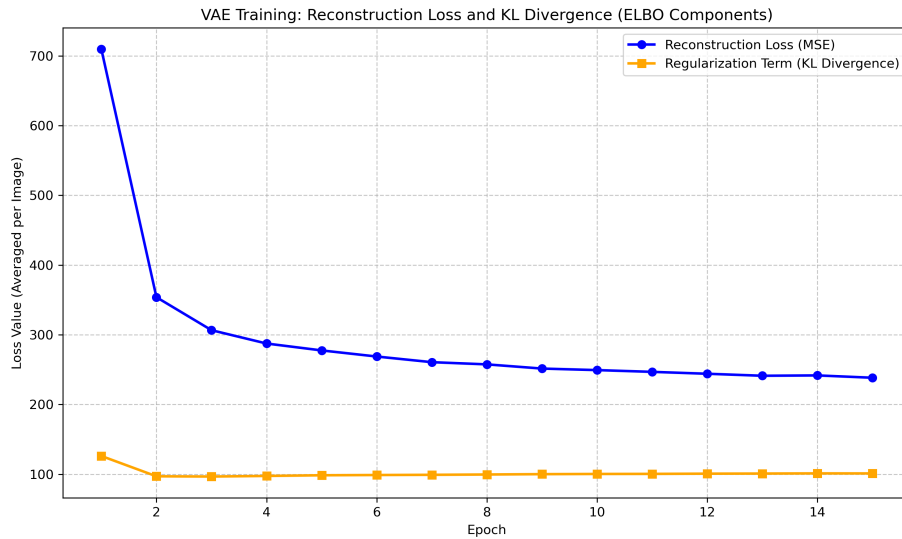
A Convolutional Variational Autoencoder (VAE) was trained on the PlantVillage dataset. The Evidence Lower Bound (ELBO) was optimized by combining reconstruction loss (MSE) and KL divergence regularization.

Result

Average losses over training epochs:

- Reconstruction Loss: decreased from ~ 710 to ~ 240
- KL Divergence: stabilized around ~ 100

Plot



Conclusion

The reconstruction loss decreases steadily, indicating improved image reconstruction quality. The KL divergence stabilizes after initial adjustment, suggesting effective latent space regularization. The balance between the two terms confirms proper ELBO optimization without signs of posterior collapse.

3.8 VAE vs. Simple DCGAN

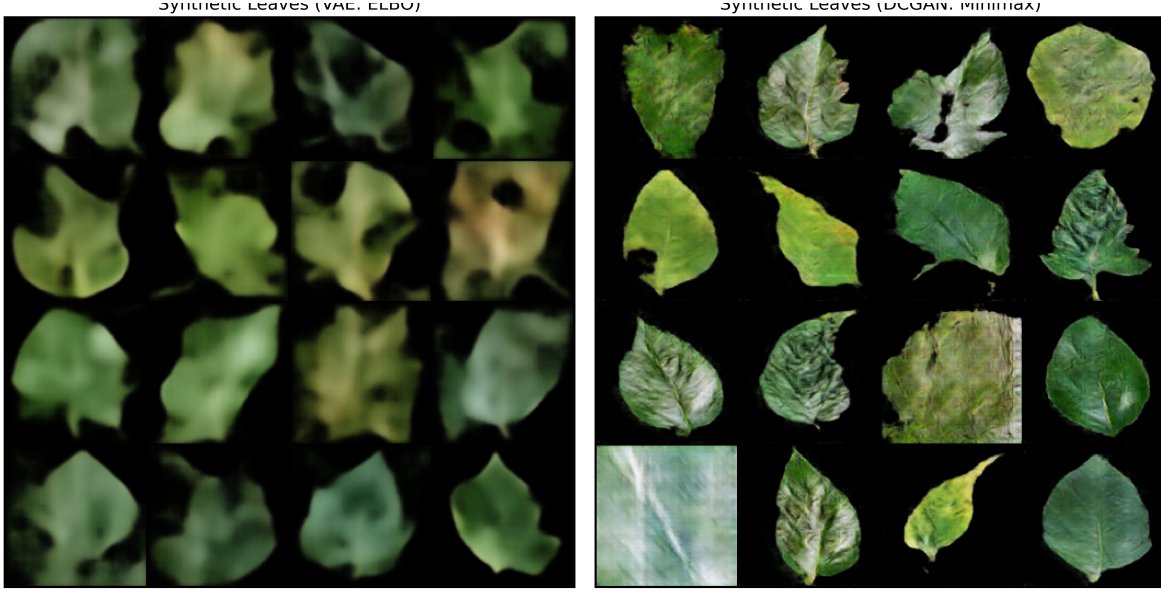
A Convolutional Variational Autoencoder (VAE) and a Deep Convolutional GAN (DCGAN) were trained on the PlantVillage dataset. A grid of 16 synthetic leaf images was generated from each model for comparison.

Result

Visual comparison of generated samples:

- VAE outputs are smooth and blurry with weak edge definition.
- DCGAN outputs are sharper with clear textures and boundaries.

Plot



Conclusion

The VAE produces blurrier images because it minimizes a pixel-wise reconstruction loss:

$$\mathbb{E}_{q(z|x)}[\|x - \hat{x}\|^2]$$

which encourages averaging over possible outputs, leading to loss of high-frequency details. In contrast, the DCGAN optimizes an adversarial objective:

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

which directly enforces realism, resulting in sharper and more visually crisp images.

3.9 Fréchet Inception Distance (FID)

The APTOS dataset was filtered to include only Class 4 images. A DCGAN was trained on this subset. A total of 500 real images and 500 generated images were passed through a frozen `inception_v3` model to extract 2048-dimensional feature representations. The FID score was computed using the standard Fréchet distance formulation.

Result

- FID Score: 436.33

Conclusion

The high FID score indicates a significant discrepancy between the distributions of real and generated images, suggesting that the GAN has not learned the data distribution effectively. Training GANs is inherently challenging due to issues such as instability, mode collapse, and sensitivity to hyperparameters. In this case, the limited dataset (Class 4 only) and model capacity likely contribute to poor generative quality, resulting in a high FID score.

3.10 Batch Normalization & Mode Collapse

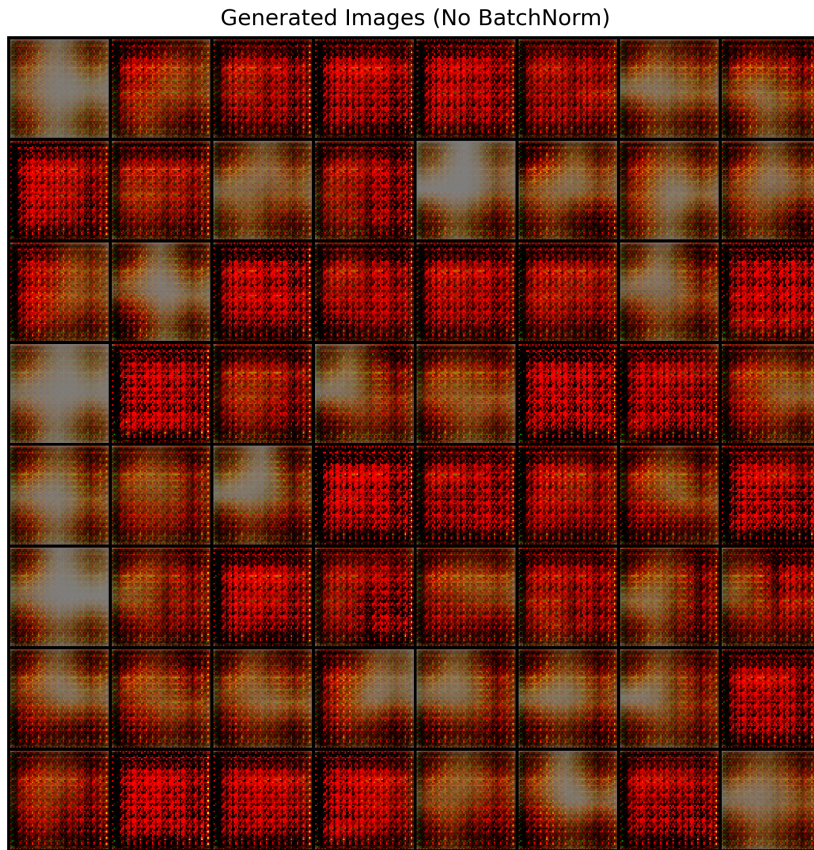
Batch Normalization layers were removed from the generator network, and the DCGAN was retrained on the APTOS dataset (Class 4). A grid of generated samples was visualized to analyze the effect on diversity and stability.

Result

Visual inspection of generated samples:

- Images exhibit repetitive patterns with low diversity.
- Strong artifacts and near-identical structures indicate mode collapse.
- Overall image quality is significantly degraded compared to the baseline GAN.

Plot



Conclusion

Batch Normalization stabilizes training by normalizing activations using batch statistics:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

which introduces coupling across samples in a batch. Removing it eliminates this regularization, allowing the generator to produce highly similar outputs:

$$G(z_1) \approx G(z_2) \approx \dots \approx x^*$$

This reduces output variance and exacerbates mode collapse. Consequently, the generator converges to a limited set of patterns, as observed in the visual grid.

4.11 InfoNCE Loss

Objective: Train a ResNet18 encoder with an MLP projection head using the InfoNCE contrastive loss on the PlantVillage dataset, without using labels.

Methodology

We follow the SimCLR framework. For each input image, two augmented views are generated using random crops, horizontal flips, and color jitter. These views form a positive pair, while all other samples in the batch act as negatives. The encoder (ResNet18) extracts representations, which are passed through a projection head (MLP) and normalized. The InfoNCE loss is computed using cosine similarity with temperature scaling, encouraging positive pairs to be closer in representation space while pushing apart negatives.

Training Details

- Encoder: ResNet18 (trained from scratch)
- Projection Head: 2-layer MLP
- Loss: InfoNCE (temperature = 0.5)
- Optimizer: Adam (lr = 10^{-3})
- Epochs: 10
- Batch Size: 128

Results

- Final Training Loss: 3.8738
- Final Validation Loss: 3.2273

Conclusion

The model successfully learns meaningful representations using only contrastive supervision. The decreasing validation loss indicates effective generalization of learned features.

4.12 Linear Evaluation Protocol

Objective

Evaluate the quality of learned representations by freezing the SimCLR encoder and training a linear classifier using labeled data.

Methodology and Training Details

The pretrained SimCLR encoder is frozen, and a single linear layer is attached as a classifier. Only this linear head is trained using labeled PlantVillage data, while the encoder weights remain unchanged. Performance is evaluated using classification accuracy on validation and test sets.

- Encoder: Frozen ResNet18 (from SimCLR)
- Classifier: Single Linear Layer
- Loss: Cross-Entropy
- Optimizer: Adam ($\text{lr} = 5 * 10^{-4}$)
- Epochs: 20 (with early stopping)
- Batch Size: 128

Results

- Validation Accuracy (Linear Eval): 94.32%
- Test Accuracy (Linear Eval): 94.55%
- Test Accuracy (Fully Supervised CNN): 94.78%

Conclusion

The linear evaluation achieves performance comparable to the fully supervised CNN, indicating that the SimCLR encoder has learned strong and transferable representations. The small accuracy gap suggests minimal loss in discriminative power despite the absence of label supervision during pretraining.

5.13 REINFORCE Implementation

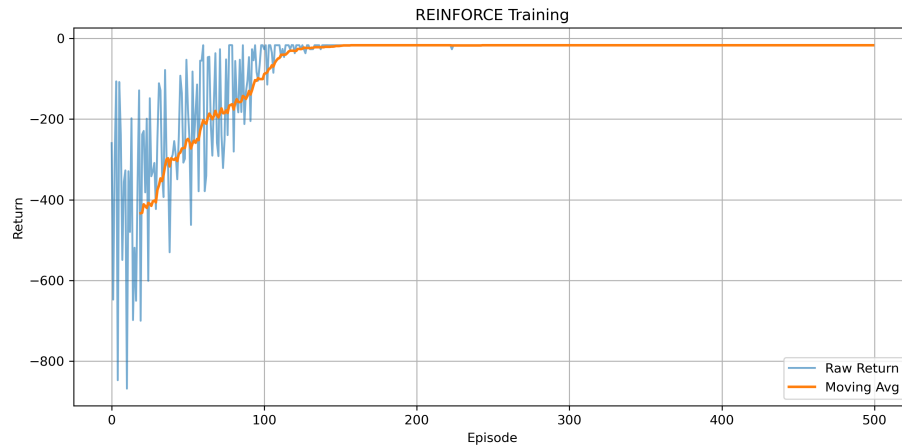
The Monte Carlo Policy Gradient (REINFORCE) algorithm was implemented to learn a policy $\pi_{\theta}(a|s)$ that outputs the probability of turning the purifier ON. The agent interacts with the environment by sampling actions and updating the policy using discounted returns and the log-derivative trick.

Result

Total episodic return at selected training stages:

- Episode 50: -52.80
- Episode 100: -16.80
- Episode 200: -16.80
- Episode 500: -16.80

Plot



Conclusion

The return improves rapidly in early episodes and stabilizes at -16.8 , indicating convergence. The learned policy consistently turns the purifier ON, avoiding large penalties and resulting in a stable but conservative strategy.